CHAPTER 1

BASIC CONCEPT

All the programs in this file are selected from

Ellis Horowitz, Sartaj Sahni, and Susan Anderson-Freed
"Fundamentals of Data Structures in C",

How to create programs

Requirements

- Describe the information that are given (input) and the results that we must produce (output)

Analysis

- bottom-up (construct a building from a generic blueprint)
- top-down (begin with the purpose that the program will serve and use this end project to divide the program into manageable segments)

How to create programs

Design:

- Data Objects (lead to the creation of abstract data types)
- Operations (require the specification of algorithms and a consideration of algorithm design strategies)

Refinement and Coding

Verification

- Program Proving (using the same technique abound in mathematics)
- Testing (require the working code and sets of test data)
- Debugging

Algorithm (演算法)

Definition

An *algorithm* is a finite set of instructions that accomplishes a particular task.

Criteria

- input (zero or more quantities that are externally supplied)
- output (at least one quantity is produced)
- definiteness: clear and unambiguous
- finiteness: terminate after a finite number of steps
- effectiveness: instruction is basic enough to be carried out

Data Type (資料型態)

Data Type
A *data type* is a collection of *objects* (物件) and a set of *operations* that act on those objects (處理這些物件的運算) .

Data Type

Abstract Data Type (抽象化資料型態)
An *abstract data type(ADT)* is a data type that is organized in such a way that the specification of the objects and the operations on the objects is separated from the representation of the objects and the implementation of the operations. (一種組織過的資料型態，組織的方式使得我們得以將物件及其運算的規格，跟物件的實際表示方法和運算的實作方法分開)

Specification vs. Implementation

Operation specification

- function name
- the types of arguments (引數)
- the type of the results

Implementation independent

*Structure 1.1: Abstract data type *Natural_Number* (p.20)

structure Natural_Number is

objects: an ordered subrange of the integers starting at zero and ending at the maximum integer (*INT_MAX*) on the computer

functions:

for all x, y $\in$ *Nat_Number*; *TRUE*, *FALSE* $\in$ *Boolean*

and where +, -, <, and == are the usual integer operations.

Nat_No Zero ( ) ::= 0

Boolean Is_Zero(x) ::= **if** (x) **return** *FALSE*
else return *TRUE*

Nat_No Add(x, y) ::= **if** ((x+y) <= *INT_MAX*) **return** x+y
else return *INT_MAX*

Boolean Equal(x,y) ::= **if** (x== y) **return** *TRUE*
else return *FALSE*

Nat_No Successor(x) ::= **if** (x == *INT_MAX*) **return** x
else return x+1

Nat_No Subtract(x,y) ::= **if** (x<y) **return** 0
else return x-y

end *Natural_Number*

::= is defined as

Measurements

Performance Analysis (machine independent)

- space complexity: storage requirement
- time complexity: computing time

Performance Measurement (machine dependent)

- such as execution time

Space Complexity (空間複雜度)

$$S(P)=C+S_P(I)$$

Fixed Space Requirements (固定的空間需求) (C)

Independent of the characteristics of the inputs and outputs (和程式的輸入輸出的數量與大小無關)

- instruction space
- space for simple variables, fixed-size structured variable, constants

Space Complexity (空間複雜度)

$$S(P)=C+S_P(I)$$

Variable Space Requirements (可變的空間需求) ($S_P(I)$)

Depend on the instance characteristic I (和要解決的實際問題有關)

- number, size, values of inputs and outputs associated with I
- recursive stack space, formal parameters, local variables, return address

***Program 1.10:** Simple arithmetic function (p.23)

```
float abc(float a, float b, float c)
{
   return a + b + b * c + (a + b - c) / (a + b) + 4.00;
 }
```

$S_{abc}(I) = 0$ (The function has fixed space requirements)

***Program 1.11:** Iterative function for summing a list of numbers (p.24)

```
float sum(float list[ ], int n)
{
  float tempsum = 0;
  int i;
  for (i = 0; i<n; i++)
     tempsum += list [i];
  return tempsum;
}
```

$S_{sum}(I) = 0$

When an array is passed as an argument to a function, C interprets it as passing the address of the first element of the array. C does not copy the array.

***Program 1.12:** Recursive function (遞迴函式) for summing a list of numbers (p.24)

```
float rsum(float list[ ], int n)
{
  if (n) return rsum(list, n-1) + list[n-1];
  return 0;
 }
```

$S_{sum}(I)=S_{sum}(n)= 12n$

Assumptions:

***Figure 1.1:** Space needed for one recursive call of Program 1.12 (p.24)

Type	Name	Number of bytes
parameter: float	list [ ]	4
parameter: integer	n	4
return address:(used internally)		4
TOTAL per recursive call		12

Time Complexity (時間複雜度)

$$T(P)=C+T_P(I)$$

Compile time (編譯時間) (C)
independent of instance characteristics (與實際輸入的特性無關)

Run (execution) time (執行時間) T_P

Time Complexity (時間複雜度)

$$T(P)=C+T_P(I)$$

Definition

A *program step* (程式步驟) is a syntactically or semantically meaningful program segment (在語法上或語意上指的是有特別意義的程式片段) whose execution time is independent of the instance characteristics (執行時間和實際輸入的特性無關).

Example (treated as one program step)

- abc = a + b + b * c + (a + b - c) / (a + b) + 4.0
- abc = a + b + c

Methods to compute the step count

Introduce global variable count into programs (將全域變數count加入到程式中)

Tabular method (表列法)

- Determine the total number of steps contributed by each statement, and find the frequency of each statement (確定每一個指令所需的步驟數，再找出每一個指令被執行的次數)
  step per execution × frequency
- Add up the contribution of all statements (將各指令的全部步驟相加)

Iterative summing of a list of numbers

***Program 1.13:** Program 1.11 with count statements (p.26)

```
float sum(float list[ ], int n)
{
   float tempsum = 0; count++; /* for assignment */
   int i;
   for (i = 0; i < n; i++) {
        count++;              /*for the for loop */
        tempsum += list[i]; count++;  /* for assignment */
   }
   count++;         /* last execution of for */
   return tempsum;
   count++;         /* for return */
}
```

2n + 3 steps

***Program 1.14:** Simplified version of Program 1.13 (p.27)

```
float sum(float list[ ], int n)
{
    float tempsum = 0;
    int i;
    for (i = 0; i < n; i++)
        count += 2;
    count += 3;
    return 0;
}
```

2n + 3 steps

Recursive summing of a list of numbers

***Program 1.15:** Program 1.12 with count statements added (p.27)

```
float rsum(float list[ ], int n)
{
        count++;        /*for if conditional */
        if (n) {
                count++;  /* for return and rsum invocation */
                return rsum(list, n-1) + list[n-1];
        }
        count++;
        return list[0];
}
```

2n+2

Matrix addition

***Program 1.16:** Matrix addition (p.28)

```
void add( int a[ ] [MAX_SIZE], int b[ ] [MAX_SIZE],
                    int c [ ] [MAX_SIZE], int rows, int cols)
{
   int i, j;
   for (i = 0; i < rows; i++)
      for (j= 0; j < cols; j++)
        c[i][j] = a[i][j] +b[i][j];
 }
```

***Program 1.17:** Matrix addition with count statements (p.29)

```
void add(int a[ ][MAX_SIZE], int b[ ][MAX_SIZE],
                    int c[ ][MAX_SIZE], int row, int cols )
{
  int i, j;
  for (i = 0; i < rows; i++){
      count++; /* for i for loop */
      for (j = 0; j < cols; j++) {
        count++; /* for j for loop */
        c[i][j] = a[i][j] + b[i][j];
        count++; /* for assignment statement */
      }
      count++;    /* last time of j for loop */
  }
  count++;        /* last time of i for loop */
}
```

2rows * cols + 2 rows + 1

***Program 1.18:** Simplification of Program 1.17 (p.29)

```
void add(int a[ ][MAX_SIZE], int b [ ][MAX_SIZE],
                    int c[ ][MAX_SIZE], int rows, int cols)
{
   int i, j;
   for( i = 0; i < rows; i++) {
      for (j = 0; j < cols; j++)
         count += 2;
      count += 2;
   }
   count++;
}
```

$2rows \times cols + 2rows + 1$

Suggestion: Interchange the loops when rows >> cols

Tabular Method

***Figure 1.2:** Step count table for Program 1.11 (p.30)

Iterative function to sum a list of numbers

steps/execution

Statement	s/e	Frequency	Total steps
float sum(float list[ ], int n)	0	0	0
{	0	0	0
float tempsum = 0;	1	1	1
int i;	0	0	0
for(i=0; i <n; i++)	1	n+1	n+1
tempsum += list[i];	1	n	n
return tempsum;	1	1	1
}	0	0	0
Total			2n+3

Recursive Function to sum of a list of numbers

***Figure 1.3:** Step count table for recursive summing function (p.31)

Statement	s/e	Frequency	Total steps
float rsum(float list[ ], int n)	0	0	0
{	0	0	0
if (n)	1	n+1	n+1
return rsum(list, n-1)+list[n-1];	1	n	n
return list[0];	1	1	1
}	0	0	0
Total			2n+2

Matrix Addition

***Figure 1.4:** Step count table for matrix addition (p.31)

Statement	s/e	Frequency	Total steps

Void add (int a[ ][MAX_SIZE] • • •)
{
int i, j;
for (i = 0; i < row; i++)
for (j=0; j< cols; j++)
c[i][j] = a[i][j] + b[i][j];
}
	0
	0
	0
	1
	1
	1
	0
		0
		0
		0
		rows+1
		rows • (cols+1)
		rows • cols
		0
			0
			0
			0
			rows+1
			rows • cols+rows
			rows • cols
			0

Total			2rows • cols+2rows+1

Asymptotic Notation (漸進符號) (O)

Definition

f(n) = O(g(n)) iff there exist positive constants c and n_0 such that $f(n) \le cg(n)$ for all n, $n \ge n_0$.

Examples

- $3n+2=O(n)$ /* $3n+2 \le 4n$ for $n \ge 2$ */
- $3n+3=O(n)$ /* $3n+3 \le 4n$ for $n \ge 3$ */
- $100n+6=O(n)$ /* $100n+6 \le 101n$ for $n \ge 10$ */
- $10n^2+4n+2=O(n^2)$ /* $10n^2+4n+2 \le 11n^2$ for $n \ge 5$ */
- $6*2^n+n^2=O(2^n)$ /* $6*2^n+n^2 \le 7*2^n$ for $n \ge 4$ */

Example

Complexity of $c_1n^2+c_2n$ and c_3n

- for sufficiently large of value, c_3n is faster than $c_1n^2+c_2n$
- for small values of n, either could be faster
  - $c_1=1$, $c_2=2$, $c_3=100$ --> $c_1n^2+c_2n \leq c_3n$ for $n \leq 98$
  - $c_1=1$, $c_2=2$, $c_3=1000$ --> $c_1n^2+c_2n \leq c_3n$ for $n \leq 998$
- break even point
  - no matter what the values of c_1, c_2, and c_3, the n beyond which c_3n is always faster than $c_1n^2+c_2n$

O(1): constant

O(n): linear

O(n^2): quadratic

O(n^3): cubic

O(2^n): exponential

O(logn)

O(nlogn)

*Figure 1.7: Function values (函數值) (p.42)

		Instance characteristic n					
Time	Name	1	2	4	8	16	32
1	Constant	1	1	1	1	1	1
$\log n$	Logarithmic	0	1	2	3	4	5
n	Linear	1	2	4	8	16	32
$n \log n$	Log Linear	0	2	8	24	64	160
n^2	Quadratic	1	4	16	64	256	1024
n^3	Cubic	1	8	64	512	4096	32768
2^n	Exponential	2	4	16	256	65536	4294967296
$n!$	Factorial	1	2	24	40326	20922789888000	26313×10^{33}

*Figure 1.8: Plot of function values (函數值的圖形) (p.43)

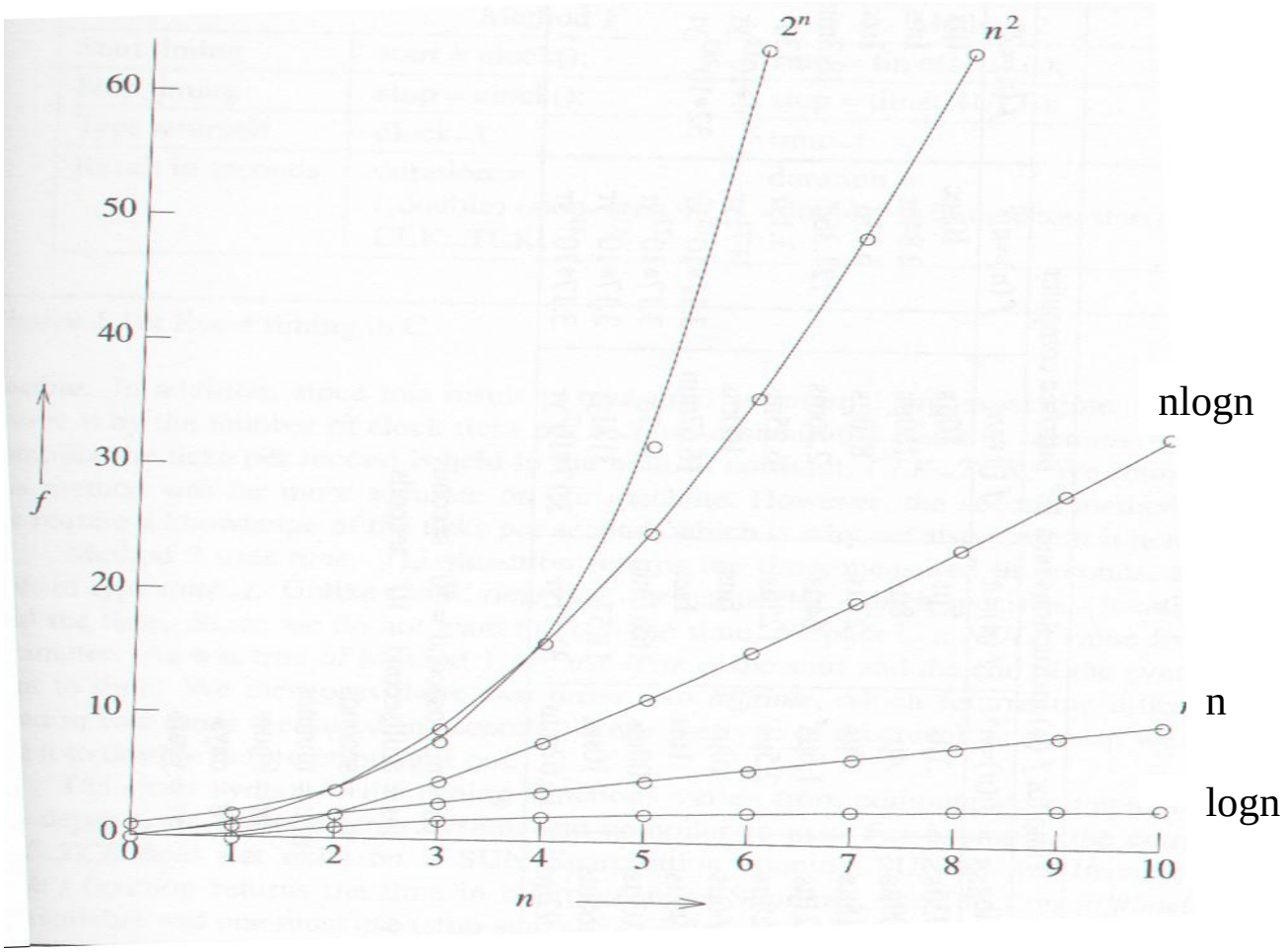


***Figure 1.9:** Times on a 1 billion instruction per second computer (在每秒鐘執行十億個指令的電腦上需要的時間) (p.44)

	Time for $f(n)$ instructions on a 10^9 instr/sec computer						
n	$f(n)=n$	$f(n)=\log_2 n$	$f(n)=n^2$	$f(n)=n^3$	$f(n)=n^4$	$f(n)=n^{10}$	$f(n)=2^n$
10	.01μs	.03μs	.1μs	1μs	10μs	10sec	1μs
20	.02μs	.09μs	.4μs	8μs	160μs	2.84hr	1ms
30	.03μs	.15μs	.9μs	27μs	810μs	6.83d	1sec
40	.04μs	.21μs	1.6μs	64μs	2.56ms	121.36d	18.3min
50	.05μs	.28μs	2.5μs	125μs	6.25ms	3.1yr	13d
100	.10μs	.66μs	10μs	1ms	100ms	3171yr	$4*10^{13}$yr
1,000	1.00μs	9.96μs	1ms	1sec	16.67min	$3.17*10^{13}$yr	$32*10^{283}$yr
10,000	10.00μs	130.03μs	100ms	16.67min	115.7d	$3.17*10^{23}$yr	
100,000	100.00μs	1.66ms	10sec	11.57d	3171yr	$3.17*10^{33}$yr	
1,000,000	1.00ms	19.92ms	16.67min	31.71yr	$3.17*10^{7}$yr	$3.17*10^{43}$yr	

μs = microsecond = 10^{-6} seconds
ms = millisecond = 10^{-3} seconds
sec = seconds
min = minutes
hr = hours
d = days
yr = years